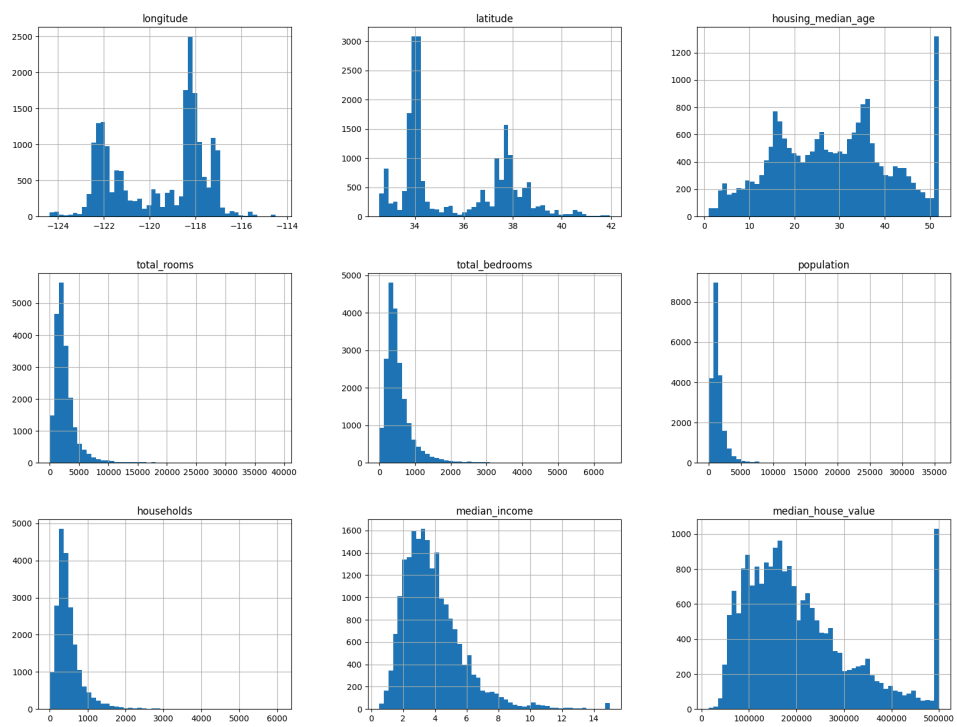
LAB 2: Machine-learning model that predicts the median housing price using the California housing price dataset.

1 Download the dataset :

```
import pandas as pd
housing=
pd.read_csv("https://media.geeksforgeeks.org/wp-content/uploads/20240319120216/housing.csv")
housing.info()
```

2. Plot the histogram of each feature(Indicate what does histogram indicate on median_income and house_median_age)

```
import matplotlib.pyplot as plt

housing.hist(bins=50, figsize=(20,15))
plt.show()
```



3.Demonstrate the process of creating a test set(write the difference between random and stratified test set)

Creating test set

```
from sklearn.model_selection import train_test_split

train_set,test_set= train_test_split(housing, test_size=0.2,
random_state=42)
```

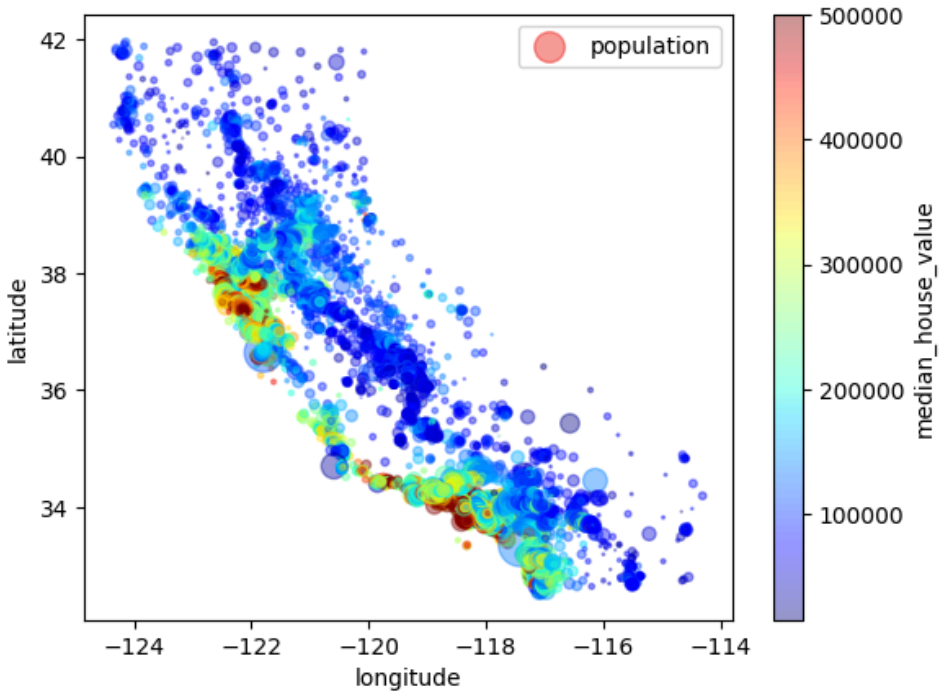
Stratified test set

```
import numpy as np

housing["income_cat"] = pd.cut(housing["median_income"],
                               bins=[0.,1.5,3.0,4.5,6.,np.inf],
                               labels=[1,2,3,4,5])

from sklearn.model_selection import StratifiedShuffleSplit

split = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)

for train_index, test_index in split.split(housing,
housing["income_cat"]):
    strat_train_set = housing.loc[train_index]
    strat_test_set = housing.loc[test_index]

for set_ in (strat_train_set, strat_test_set):
    set_.drop("income_cat", axis=1, inplace=True)
```

4. List the geographical features from the dataset and plot a graph to Visualize Geographical Data(what does the graph indicate w.r.t housing prices and location)

```
housing.plot(kind="scatter", x="longitude", y="latitude", alpha=0.4,
             s=housing["population"]/100, label="population",
             c="median_house_value", cmap=plt.get_cmap("jet"),
colorbar=True)
```

```
plt.legend()
plt.show()
```



5. Plot a graph to show features correlation with housing price. Which feature corelates to the maximum. Plot the graph for that with housing price and analyze what the graph indicate

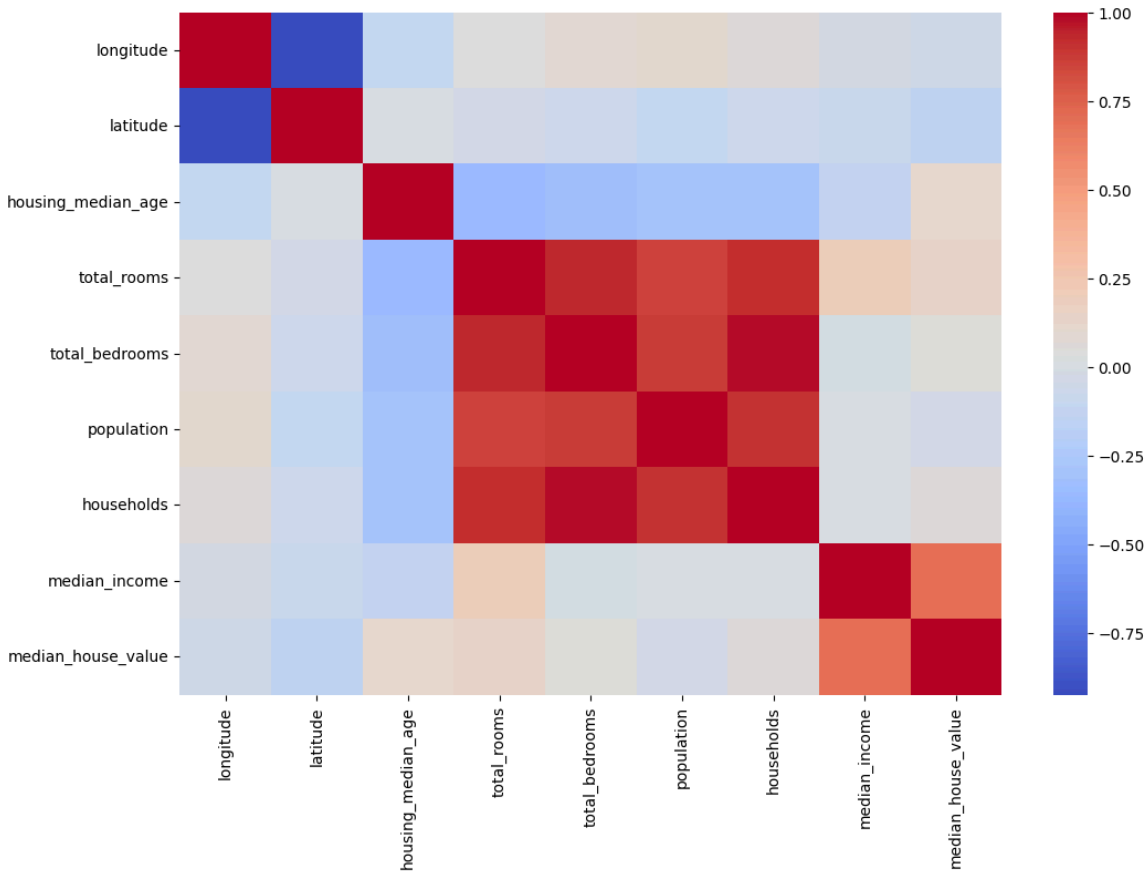
correlation

```
corr_matrix = housing.corr(numeric_only=True)
print(corr_matrix["median_house_value"].sort_values(ascending=False))
```

```
median_house_value    1.000000
median_income         0.688075
total_rooms           0.134153
housing_median_age    0.105623
households            0.065843
total_bedrooms        0.049686
population           -0.024650
longitude            -0.045967
latitude             -0.144160
Name: median_house_value, dtype: float64
```

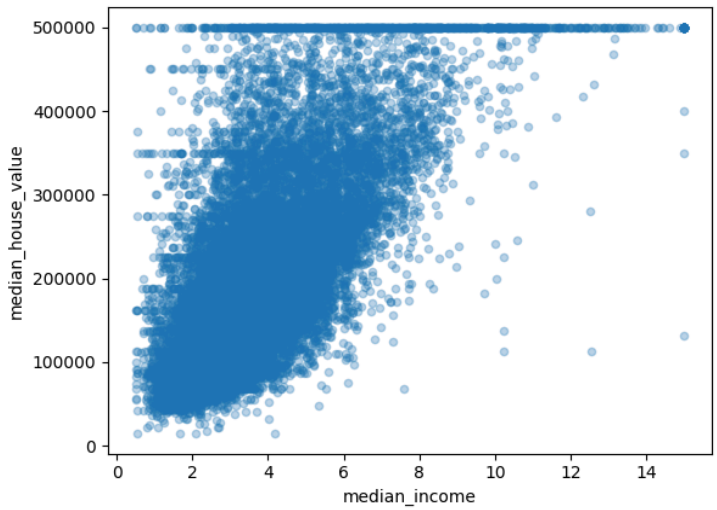
Correlation Heatmap

```
import seaborn as sns

plt.figure(figsize=(12,8))
sns.heatmap(corr_matrix, annot=False, cmap="coolwarm")
plt.show()
```



Plot median_income vs house value

```
housing.plot(kind="scatter", x="median_income", y="median_house_value",
alpha=0.3)
plt.show()
```



6. List the features that could be combined to improve correlation and plot again to see if correlation has improved

```
housing["rooms_per_household"] = housing["total_rooms"] /
housing["households"]
housing["bedrooms_per_room"] = housing["total_bedrooms"] /
housing["total_rooms"]
housing["population_per_household"] = housing["population"] /
housing["households"]

corr_matrix = housing.corr(numeric_only=True)
print(corr_matrix["median_house_value"].sort_values(ascending=False))
```

```
median_house_value          1.000000
median_income               0.688075
rooms_per_household         0.151948
total_rooms                 0.134153
housing_median_age          0.105623
households                  0.065843
total_bedrooms              0.049686
population_per_household   -0.023737
population                 -0.024650
longitude                  -0.045967
latitude                   -0.144160
bedrooms_per_room          -0.255880
Name: median_house_value, dtype: float64
```

7. List the features that needs to be cleaned and demonstrate the process of cleaning

Method 1: Remove rows

```
housing.dropna(subset=["total_bedrooms"])
```

Method 2: Fill with Median

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")

housing_num = housing.drop("ocean_proximity", axis=1)
imputer.fit(housing_num)

housing_num_imputed = imputer.transform(housing_num)
```

8. Is there any categorical data that needs to be converted to numerical? If so explain the method used to convert and code the same and show the output.

```
from sklearn.preprocessing import OneHotEncoder

housing_cat = housing[["ocean_proximity"]]

encoder = OneHotEncoder()
housing_cat_1hot = encoder.fit_transform(housing_cat)

print(encoder.categories_)
print(housing_cat_1hot.toarray())
```

```
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
      dtype=object)]
[[0. 0. 0. 1. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 1. 0.]
 ...
 [0. 1. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 1. 0. 0. 0.]]
```